

From Fixed Keys to Readable Schemas: Small Language Models for Vehicle Agent Function Calls

Hamed Jafarzadeh Asl

Huawei Noah’s Ark Lab, Canada
hamed.jafarzadeh.asl@h-partners.com

Yuanhao Yu

Huawei Noah’s Ark Lab, Canada
yuanhao.yu@huawei.com

Vahid Partovi Nia

Huawei Noah’s Ark Lab and Polytechnique Montreal, Canada
vahid.partovinia@huawei.com

Abstract

In-vehicle assistants must translate natural-language requests into accurate vehicle function calls under strict memory and latency constraints, making small language models (SLMs) attractive for on-device deployment. For such models, a key design choice is how the available function surface is presented. Two approaches are to represent each function with a dedicated Functional Token (FT) or provide function schemas directly in the prompt. FTs enable compact inference but are restricted to functions learned during training, whereas Schema-in-Prompt (SIP) can generalize to unseen functions at the cost of longer prompts and higher inference overhead. We introduce a benchmark of 9,822 single-turn examples spanning 79 vehicle functions derived from Android Automotive, including held-out functions and requests requiring refusal. We compare both approaches under matched fine-tuning across four SLMs from 270M to 1.7B parameters. On functions seen during training, scaling provides limited benefit: the 270M model can match the 1.7B model, while the strongest overall performance occurs at 0.6B. On held-out functions, FT achieves zero accuracy by construction, whereas SIP generalizes and improves substantially with scale. On out-of-scope requests, FT can invoke an unavailable function it was trained to emit, while SIP more reliably refuses based on the functions offered. This flexibility comes with higher memory use and latency. Our theoretical analysis explains how SIP enables generalization and why longer schema contexts increase inference cost. Overall, function-surface representation, rather than model scale alone, determines the capabilities and failure modes of SLM-based vehicle function calling.

1 Introduction

Reliable vehicle function calling requires an in-vehicle assistant to map a natural-language request to the correct vehicle action, including the appropriate function and arguments, or to refuse when the request cannot be satisfied. For example, “move my seat one step forward” should invoke the corresponding seat-control function with the correct arguments, as illustrated in Figure 1 (left). Unlike conventional text generation, an incorrect prediction may directly actuate physical hardware. Moreover, the available function surface can vary across vehicle models, model years, and trim levels, while deployment is constrained by memory, latency, and connectivity. These conditions make small language models (SLMs) attractive for on-device execution, but raise an important design question: *how should the available functions be represented to a small model?*

We study two representations with fundamentally different properties. A *Functional Token* (FT) approach (Chen and Li, 2026; Sowti Khiabani et al., 2025) assigns each supported function a dedicated vocabulary token learned during training. This representation is compact at inference time, but its task-level output space is tied to functions encoded during training, so supporting a new function requires adapting the representation and model. In contrast, our *Schema-in-Prompt* (SIP)

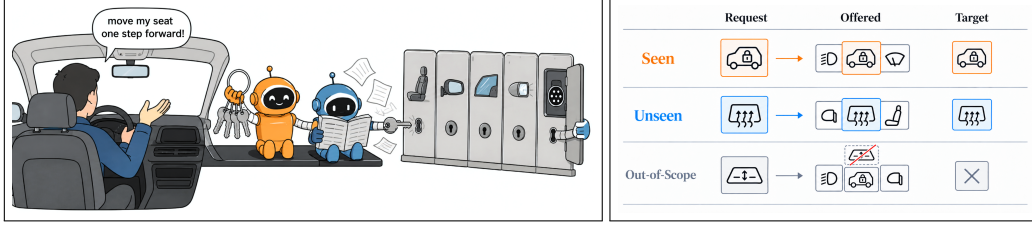


Figure 1: Function-calling representations and evaluation splits. Left: Functional Token represents functions as fixed vocabulary entries, while Schema-in-Prompt conditions on offered function schemas at inference time. Right: *Seen* functions appear as training targets, *Unseen* functions are withheld as targets, and *Out-of-Scope* requests have no suitable offered function and require refusal.

approach supplies the available function schemas in the input context. The model interprets these schemas at inference time and selects among the functions offered for the current vehicle. SIP can therefore admit functions never used as training targets, at the cost of longer prompts and additional inference computation. The resulting question is not simply whether a larger SLM produces better function calls, but how the representation of the function surface affects generalization, refusal, and deployment cost.

Existing function-calling work explores several ways to adapt language models for tool use, including API selection, call construction, and invocation decisions. For compact deployment, Octopus v2 assigns functions dedicated vocabulary entries (Chen and Li, 2026), while other work investigates function masking, retrieval, low-rank adaptation, dynamic function selection, and action-specialized models (Lin et al., 2024; Erdogan et al., 2024; Paramanayakam et al., 2024; Zhang et al., 2024). The closest prior work for in-vehicle function calling prunes a general language model and represents eight vehicle functions using dedicated vocabulary tokens (Sowti Khiabani et al., 2025). Its proprietary function set and private dataset prevent a controlled numerical comparison, while the lack of a public vehicle-specific benchmark makes it difficult to evaluate how alternative representations behave as the supported function surface changes. Language interfaces for vehicles have also been studied in adjacent settings. Talk2Car grounds passenger commands in street scenes (Deruyttere et al., 2019), DriveLM studies visual question answering for driving (Sima et al., 2024), and Talk2Drive translates natural-language commands into driving controls (Cui et al., 2023). These works primarily address perception, reasoning, planning, or driving control. We instead study single-turn selection and refusal over documented in-vehicle functions, focusing on how SLMs behave when the available function surface differs from the functions they were trained to execute.

We construct a vehicle function-calling benchmark derived from Android Automotive, separating functions observed as training targets, functions withheld as targets, and requests for which no suitable offered function exists. We use this benchmark to compare FT and SIP under matched training settings across SLMs from 270M to 1.7B parameters. Our results show that the two representations behave similarly on functions observed during training but diverge when the available function surface changes, revealing a trade-off between the flexibility of schema-conditioned function selection and its inference overhead. Our contributions are:

- (i) a reproducible vehicle function-calling benchmark derived from a documented Android Automotive function surface, with separate Seen, Unseen, and Out-of-Scope evaluation;
- (ii) a controlled comparison of Functional Token and Schema-in-Prompt representations across SLMs from 270M to 1.7B parameters, including their function-selection, refusal, latency, and memory behavior;
- (iii) a theoretical analysis explaining the difference in task-level output support between the two representations, how schema-mediated generalization to Unseen Functions becomes possible, and why this flexibility introduces additional inference cost.

2 Vehicle Function-Calling Benchmark

We construct a benchmark for Single-Turn Function Calling from Android Automotive VehiclePropertyIds (Android Open Source Project, 2026). These specifications describe controllable vehicle properties together with the locations where they apply and the values they accept.

We convert the controllable properties into function schemas while preserving these constraints. For example, when a control can act on different seats or windows, the vehicle location is represented as an argument. Documented numerical ranges and enumerated states define the legal argument values. Read-only properties that only report vehicle state are excluded. This produces a fixed, externally specified Function Surface rather than a collection of functions invented for the benchmark.

Generation and validation. For examples whose correct output is a Vehicle Function Call, we first select a function and sample legal argument values from its schema. The resulting call is validated programmatically before any natural-language request is generated. A language model is then asked to express the validated call as a user request. The target function and arguments are therefore defined independently of the generated wording. We separately evaluate whether that wording is suitable for the benchmark. Generated requests are audited for *Target Leakage*, where the wording directly exposes the target function identifier, screened by a second model from a different family, and manually inspected through a stratified *Verified Slice*. The Verified Slice contains 217 examples across 42 strata. Of these, 100.0% are judged faithful to their targets and 99.1% are judged unambiguous. Appendix A gives the complete construction and validation procedure.

Evaluation design. Each benchmark example is associated with an *offered set*, which defines the functions available for that example. *Seen Functions* occur as targets during training. *Unseen Functions* never occur as training targets. Related functions are withheld together rather than holding out a single function while training on close counterparts from the same vehicle-control family. Their schemas may still appear as non-target candidates during training. SIP can therefore learn how to interpret offered schemas without being trained to select these held-out functions as answers.

Out-of-Scope Requests require refusal because no suitable function is available. We construct two cases. Some requests ask for functionality absent from the complete Function Surface. Others request a valid function that is deliberately omitted from the offered set for that example. The second case tests whether the model recognizes that a function is unavailable rather than only identifying the operation requested by the user.

Each offered set contains 4 to 24 functions, with median 14. Seen and Unseen schemas are mixed so that the composition of the offered set does not reveal the evaluation split or identify the answer by elimination. We also prevent equivalent requests from appearing on both sides of the training and evaluation partition. For globally unsupported functionality, different phrasings of the same underlying request type are assigned to the same partition. Table 1 summarizes the corpus with 9,822 examples, and the partition of its 79-function surface into Seen, Unseen, and Out-of-Scope functions. The 18 Unseen Functions are withheld as training targets, while the five Out-of-Scope functions are valid functions withheld from every offered set. Individual examples may additionally omit an otherwise available function from their own offered set.

Split	Examples	Training	Evaluation	Surface functions
Seen	5,521	4,975	546	56
Unseen	2,146	0	2,146	18
Out-of-Scope	2,155	1,622	533	5
Total	9,822	6,597	3,225	79

Table 1: Benchmark composition and Function Surface partition. *Surface functions* reports how the 79 functions are assigned to Seen, Unseen, and Out-of-Scope groups, not the number of distinct functions referenced by the examples in each row.

3 Theoretical Analysis of Function-Surface Representation

We theoretically characterize the distinction between Functional Token (FT) and Schema-in-Prompt (SIP) representations through their task-level output support. Let T denote the set of functions assigned dedicated Functional Tokens during training, and let $\perp$ denote refusal. Under FT, the admissible task-level output set is fixed, denoted as $\mathcal{Y}_{\text{FT}} = T \cup \{\perp\}$. Therefore, a function $f \notin T$ cannot be recovered as an exact FT prediction unless the output representation is extended and

the model is adapted to the new function. SIP instead derives its admissible function set from the functions O_i offered for task instance i , denoted as $\mathcal{Y}_{\text{SIP},i} = O_i \cup \{\perp\}$. A function withheld as a training target can therefore remain a valid prediction at inference time when its schema is included in O_i . This difference also affects refusal behavior. SIP explicitly conditions on the functions available for the current task instance, whereas the evaluated FT formulation does not expose the row-specific offered set to the model.

The additional flexibility of SIP comes with a computational trade-off. Because SIP includes the serialized function schemas in the input context, its sequence length grows with the number and size of the offered schemas. For Transformer-based models, this increases prompt-processing cost and key-value cache requirements relative to FT, providing a theoretical basis for the latency and memory differences evaluated empirically in our device-proxy experiments. Appendix B provides the full theoretical development, including the task-level support argument, schema-mediated generalization, refusal behavior, and inference-cost analysis.

4 Experimental Setup

Models and adaptation. We evaluate four compact open-weight models: Gemma 3-270M, FunctionGemma-270M, Qwen3-0.6B, and Qwen3-1.7B. The two 270M models provide a same-scale comparison of general-purpose and function-calling-specialized pretraining, while the Qwen3 pair provides a within-family comparison of model capacity. Each model is adapted separately with Functional Token (FT) and Schema-in-Prompt (SIP) representations using the same training corpus. The 270M and 0.6B configurations use full fine-tuning, while Qwen3-1.7B uses LoRA in the main grid. Appendix C.6 examines full fine-tuning for Qwen3-1.7B SIP and FunctionGemma’s native function-calling prompt format.

Training and evaluation. Matched FT and SIP configurations use the same training examples and optimization recipe, with three independent seeds per configuration. We report exact Vehicle Function Call accuracy for Seen and Unseen Functions and correct-refusal accuracy for Out-of-Scope Requests. For Unseen Functions, we additionally report *Execution*, the rate at which the model produces any valid Vehicle Function Call; the gap between Execution and exact accuracy therefore captures incorrect execution. For Out-of-Scope Requests, *Over-trigger* measures the rate at which the model produces a Vehicle Function Call instead of refusing. Optimization, decoding, parsing, metric, and aggregation details are provided in Appendix C.

5 Results

The experiments reveal a consistent division between representation and model capacity. Performance on functions observed during training is already high across the evaluated SLMs, while behavior under a changing Function Surface depends strongly on whether functions are encoded as learned outputs or supplied through schemas at inference time. Figures 2 and 3 summarize these effects; the complete numerical grid is reported in Appendix C.3.

Seen Function performance is already near saturation. Figure 2 (left) shows that both FT and SIP achieve high accuracy on Seen Functions across the evaluated models. Qwen3-0.6B obtains the highest mean Seen accuracy in the main grid, while the 270M configurations remain close to the larger models. For SIP, paired tests detect no significant difference between FunctionGemma-270M and Qwen3-1.7B on Seen Functions in any seed. This does not establish equivalence, but provides no evidence that the substantially larger configuration improves Seen Function performance in this comparison. Because the evaluated models span different families, we do not interpret the overall ordering as a general scaling law.

Representation determines whether Unseen Functions can be recovered. The distinction becomes fundamental when the requested function was withheld as a training target. As shown in Figure 3 (left), FT achieves zero exact accuracy on Unseen Functions because no trained Functional Token represents these targets, consistent with the task-level output-support analysis in Section 3 and Appendix B.1. SIP instead admits an Unseen Function whenever its schema is included in the offered set and achieves nonzero generalization across all evaluated models. Within Qwen3, the

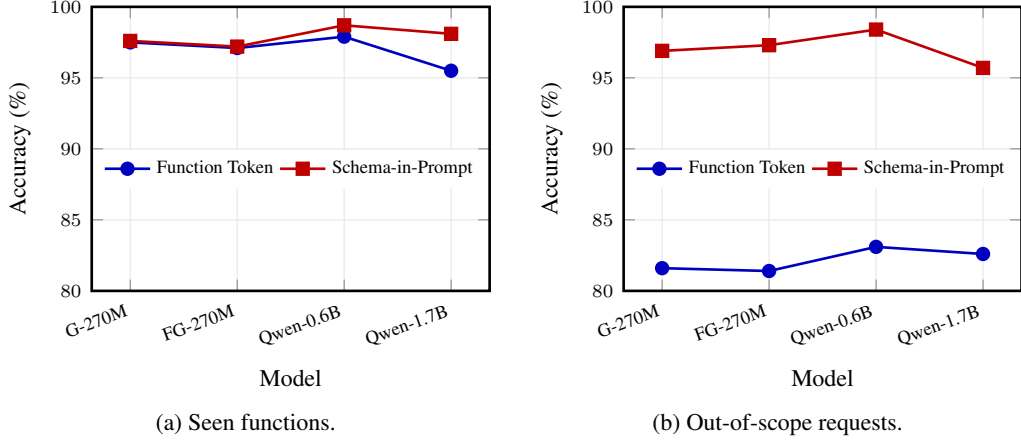


Figure 2: Seen Function and Out-of-Scope accuracy of the Function Token (FT) and Schema-in-Prompt (SIP) approaches. FT and SIP perform similarly on Seen Functions (left), whereas SIP consistently achieves higher correct-refusal accuracy on Out-of-Scope Requests (right). FG denotes FunctionGemma-270M and G denotes Gemma 3-270M.

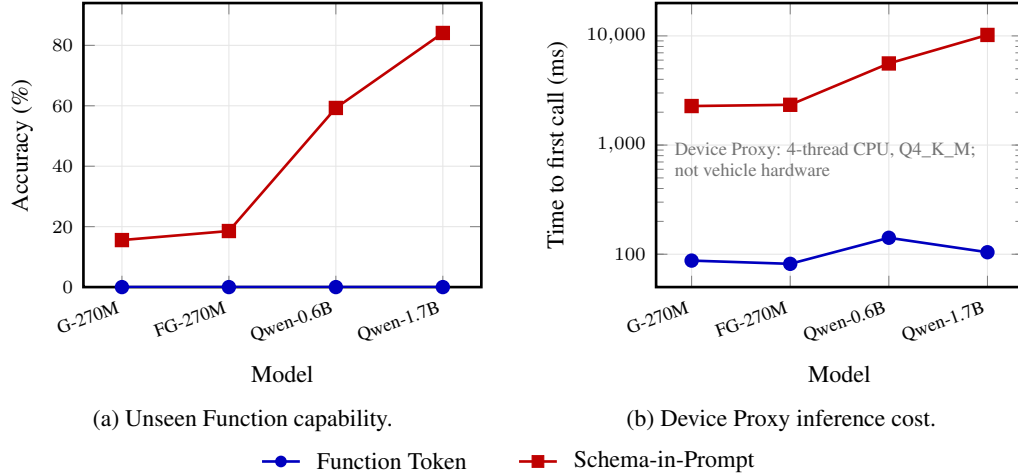


Figure 3: Unseen Function capability and inference cost. Left, mean Unseen Function accuracy across three seeds. Right, median Device Proxy time to first call across three measurement runs. FG denotes FunctionGemma-270M and G denotes Gemma 3-270M. Device Proxy measurements are matched host measurements and do not represent latency on vehicle hardware.

1.7B configuration substantially improves Unseen accuracy over 0.6B. At 270M, FunctionGemma also outperforms Gemma 3 on every seed, with a significant paired difference in each seed. Model capacity and prior function-calling specialization therefore improve schema interpretation, but only SIP provides the representational mechanism required to select a function that was never a training target.

Execution exposes the consequence of unsupported targets. Unseen accuracy alone does not reveal whether an unsuccessful model refuses or attempts an incorrect action. Table 2 therefore reports *Execution*, the rate of producing any valid Vehicle Function Call on an Unseen request, alongside exact accuracy. For FT, Unseen accuracy is zero, so any such execution is necessarily incorrect. For SIP, the gap between Execution and accuracy measures executions that do not exactly match the target call. This distinction is important for vehicle control: incorrect execution can actuate an unintended function or arguments, whereas refusal leaves the request unexecuted.

SIP better conditions refusal on the available Function Surface. Figure 2 (right) shows a consistent FT-SIP separation on Out-of-Scope Requests. The difference is concentrated in requests

Representation	Model	Unseen Functions		Out-Of-Scope
		Accuracy	Execution	Over-trigger
Functional Token	Qwen3-1.7B	0.0	6.0	17.4
	Qwen3-0.6B	0.0	0.4	16.9
	FunctionGemma-270M	0.0	1.5	18.4
	Gemma 3-270M	0.0	1.5	18.4
Schema-in-Prompt	Qwen3-1.7B	84.1	87.9	4.3
	Qwen3-0.6B	59.3	61.7	1.6
	FunctionGemma-270M	18.6	21.6	2.7
	Gemma 3-270M	15.6	18.2	3.1

Table 2: Behavioral rates (%) averaged over three seeds. For Unseen Functions, *Accuracy* requires an exact match to the target Vehicle Function Call, whereas *Execution* is the rate of producing an executable call to an available function. Their difference measures non-exact executions. For Out-of-Scope Requests, *Over-trigger* is the rate of producing a Vehicle Function Call instead of refusing.

for valid vehicle functions that are absent from the offered set. When the unavailable function has a trained Functional Token, FT frequently produces a call despite the function being unavailable, whereas SIP usually refuses. When the unavailable function is not readily producible by either representation, both methods refuse reliably. This decomposition supports the representational explanation: SIP observes the offered Function Surface for the current example, while the evaluated FT formulation does not. Full results are reported in Appendix C.5.

Schema-mediated generalization introduces an inference cost. The additional capability of SIP comes from placing the offered schemas in the input context and therefore requires more prompt processing. Figure 3 (right) shows the corresponding Device Proxy time-to-first-call difference. Across matched model pairs, SIP requires 26–98 $\times$ the Device Proxy time to first call and 2.4–4.2 $\times$ the peak resident memory of FT, while adding 1,785 to 1,905 prompt tokens. These measurements agree with the sequence-length analysis in Appendix B. Full measurements and implementation-specific effects are reported in Appendix C.7.

Appendix C.6 further tests Qwen3-1.7B adaptation method and FunctionGemma prompt format. Neither ablation changes the central result: FT provides the more compact representation, while SIP supports generalization and availability-aware refusal when the Function Surface changes.

6 Conclusion

Reliable in-vehicle function calling requires a small on-device model to map driver requests to the correct Vehicle Function Call, or refuse them when appropriate. This must be done under strict deployment constraints, even when the available Function Surface changes after training. We compare two ways to represent this surface: Functional Token (FT), which encodes functions as learned vocabulary entries, and Schema-in-Prompt (SIP), which provides function schemas at inference time. Both approaches perform strongly on Seen Functions, with limited benefit from additional model capacity in this regime. Their behavior diverges when the Function Surface changes. FT cannot correctly predict Unseen Functions outside its learned output space and may instead substitute an incorrect known function. SIP generalizes substantially better to Unseen Functions and more reliably handles Out-of-Scope Requests. Our theoretical analysis explains this difference through the task-level output support of the two representations.

This flexibility comes with a deployment cost. SIP requires longer prompts and higher Device Proxy latency and memory use, while FT remains more compact at inference time. Function Surface representation is therefore a deployment decision, not only a modeling choice. FT is better suited to stable Function Surfaces under tight inference constraints, whereas SIP is better suited to systems that must accommodate changing functions without modifying the learned output representation. Our benchmark is synthetic, English-only, and single-turn, and efficiency is measured on a Device Proxy rather than vehicle hardware. Future work should extend the evaluation to natural driver requests, multilingual and multi-turn interaction, and measurements on vehicle hardware.

References

- Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. (2023). What learning algorithm is in-context learning? investigations with linear models. In *International Conference on Learning Representations*.
- Android Open Source Project (2026). VehiclePropertyIds. Android Automotive API reference. Accessed 2026-08-24.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*.
- Chen, W. and Li, Z. (2026). Octopus v2: On-device language model for super agent.
- Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.
- Cui, C., Yang, Z., Zhou, Y., Ma, Y., Lu, J., Li, L., Chen, Y., Panchal, J., and Wang, Z. (2023). Personalized autonomous driving with large language models: Field experiments.
- Deruyttere, T., Vandenhende, S., Grujicic, D., Van Gool, L., and Moens, M.-F. (2019). Talk2Car: Taking control of your self-driving car. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*.
- Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C., Anumanchipalli, G., Keutzer, K., and Gholami, A. (2024). Tinyagent: Function calling at the edge.
- Geifman, Y. and El-Yaniv, R. (2017). Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*.
- Jeon, H. J., Lee, J. D., Lei, Q., and Van Roy, B. (2024). An information-theoretic analysis of in-context learning.
- Lin, Q., Wen, M., Peng, Q., Nie, G., Liao, J., Wang, J., Mo, X., Zhou, J., Cheng, C., Zhao, Y., Wang, J., and Zhang, W. (2024). Hammer: Robust function-calling for on-device language models via function masking.
- Liu, J., Shen, D., Zhang, Y., Dolan, B., Carin, L., and Chen, W. (2022). What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*.
- Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. (2022). Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. (2022). Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.
- Paramanayakam, V., Karatzas, A., Anagnostopoulos, I., and Stamoulis, D. (2024). Less is more: Optimizing function calling for LLM execution on edge devices.
- Rubin, O., Herzig, J., and Berant, J. (2022). Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Shalev-Shwartz, S. and Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.

- Sima, C., Renz, K., Chitta, K., Chen, L., Zhang, H., Xie, C., Beißwenger, J., Luo, P., Geiger, A., and Li, H. (2024). DriveLM: Driving with graph visual question answering. In *European Conference on Computer Vision*.
- Sowti Khiabani, Y., Atif, F., Hsu, C., Stahlmann, S., Michels, T., Kramer, S., Heidrich, B., Sarfraz, M. S., Merten, J., and Tafazzoli, F. (2025). Optimizing small language models for in-vehicle function-calling.
- Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. (2022). Efficient transformers: A survey.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. (2017). Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. (2023). Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*.
- Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. (2021). An explanation of in-context learning as implicit Bayesian inference.
- Zhang, J., Lan, T., Zhu, M., Liu, Z., Hoang, T., Kokane, S., Yao, W., Tan, J., Prabhakar, A., Chen, H., Liu, Z., Feng, Y., Awalgaonkar, T., Murthy, R., Hu, E., Chen, Z., Xu, R., Niebles, J. C., Heinecke, S., Wang, H., Savarese, S., and Xiong, C. (2024). xlam: A family of large action models to empower ai agent systems.

A Benchmark Construction and Validation

This appendix expands the benchmark construction described in Section 2. We describe how documented Android Automotive controls are converted into function schemas, how structured calls and natural-language requests are generated, how the benchmark partitions are constructed, and how the resulting synthetic corpus is validated.

A.1 Function Surface Construction

The Function Surface is derived from Android Automotive `VehiclePropertyIds` (Android Open Source Project, 2026). We retain properties that correspond to controllable vehicle functionality and exclude properties that only report vehicle state. Each retained control is converted into a callable schema using constraints provided by the documented interface. When a control applies to several vehicle locations, such as different seats or windows, the location is represented explicitly as an argument. Documented numerical ranges, units, Boolean states, and enumerated modes determine the admissible values of the remaining arguments. The resulting schemas therefore preserve the structure and constraints of the source interface while expressing them in a form suitable for Vehicle Function Calling. The Function Surface is fixed before corpus generation, and the same definitions are used during both data construction and evaluation. The Function Surface contains 79 functions partitioned into Seen, Unseen, and Out-of-Scope groups. Seen and Unseen Functions may serve as Vehicle Function Call targets, with 18 Unseen Functions withheld as training targets. The Out-of-Scope group contains five valid functions that are deliberately withheld from every offered set, so requests for these functions require refusal. Out-of-Scope Requests are not limited to these five. An ordinary Seen or Unseen function may also be omitted from a single example’s offered set, which makes it unavailable for that example alone. This second construction is what allows the refusal analysis in Appendix C.5 to separate unavailable functions that the FT representation holds a token for from those it does not.

A.2 Representative Examples

Figure 4 shows representative Seen, Unseen, and Out-of-Scope examples. Each row contains a natural-language request, the offered functions for that example, and the expected Vehicle Function Call or refusal. The examples also illustrate why the offered set is part of the task definition. Nearly equivalent requests may require different outputs when the appropriate function is available in one offered set but absent from another.

A.3 Label-First Generation

Examples with a Vehicle Function Call as the correct output are constructed by generating the structured target before the natural-language request. We first select a function from the Function Surface and sample argument values that satisfy its schema. The resulting Vehicle Function Call is checked programmatically for a valid function identity, required arguments, argument types, permitted values, numerical ranges, and applicable vehicle area. Only after this validation is the structured call provided to the corpus-writing model, which generates a natural-language request intended to express the same operation. We use Llama-3.3-70B-Instruct as the corpus writer. The target call therefore does not depend on a language model inferring a label from previously generated text. This ordering separates structured validity from language quality. Programmatic validation establishes that the target itself is a legal Vehicle Function Call. It does not establish that the generated request describes that call faithfully. We assess these language-level properties separately in Appendix A.7. The generation prompt provides the intended operation, vehicle location, and semantic interpretation of argument values. It also varies the requested speaking style to reduce repeated templated phrasing. Generated requests are normalized and deduplicated before partitioning. This prevents repeated or punctuation-equivalent requests from receiving disproportionate weight or appearing independently in training and evaluation.

A.4 Unseen Functions and Offered Sets

An Unseen Function is a function that never appears as the correct target of a training example. We withhold related functions together rather than holding out an isolated function while training

Seen	Request: <i>"Lock the boot."</i> Label: <code>door_set_lock(area="DOOR_REAR", on=true)</code> Offered: <code>seat_move_lumbar_vertical, mirror_set_heat, rear_fog_lights_set,</code> <code>front_fog_lights_set, seat_set_ventilation, seat_move_lumbar_side,</code> <code>hvac_set_defroster, door_set_lock, hvac_set_max_defrost, ev_set_charge_limit,</code> <code>seat_move_cushion_side_support, seat_move_depth, steering_wheel_set_heat,</code> <code>turn_signal_set</code> The target function was a training target. 0-003954
Unseen	Request: <i>"Turn on the rear demister."</i> Label: <code>hvac_set_defroster(area="WINDOW_REAR_WINDSHIELD", on=true)</code> Offered: <code>seat_move_lumbar_side, rear_fog_lights_set,</code> <code>headlights_set, front_fog_lights_set, ev_set_charging,</code> <code>adas_set_blind_spot_warning, seat_move_headrest_height,</code> <code>hvac_set_fan_direction, cruise_control_set_time_gap, mirror_move_horizontal,</code> <code>cruise_control_set_target_speed, ev_set_charge_current_limit,</code> <code>ev_set_charge_limit, seat_move_lumbar_fore_aft, hvac_set_defroster,</code> <code>cruise_control_set_enabled, seat_set_easy_access, window_set_child_lock</code> The schema is offered; the function was never a training target. 0-011994
Out-of-Scope	Request: <i>"Lock the trunk."</i> Label: <i>decline</i> Offered: <code>set_head_up_display, mirror_move_horizontal,</code> <code>ev_set_charging, cruise_control_set_enabled, turn_signal_set,</code> <code>steering_wheel_set_heat, mirror_move_vertical, seat_move_lumbar_side,</code> <code>ev_set_charge_current_limit, hvac_set_power, hvac_set_max_defrost,</code> <code>ev_set_charge_port, seat_move_headrest_height, seat_move_lumbar_vertical,</code> <code>ev_set_charge_limit, window_set_child_lock, front_fog_lights_set,</code> <code>hvac_set_defroster, hvac_set_fan_speed, cruise_control_set_target_speed,</code> <code>seat_move_lumbar_fore_aft, adas_set_cross_traffic_monitoring</code> The vehicle has this function, but it is absent from the offered list. Only the offered list makes the correct answer a refusal. Note the first row: the same function, requested almost the same way, is a correct call there and a correct refusal here. 0-019056

Figure 4: Representative benchmark examples. Seen and Unseen examples require a Vehicle Function Call when the appropriate function is available. An Out-of-Scope example requires refusal when no suitable function is present in the offered set.

on closely related functions from the same control family. This reduces the possibility that Unseen evaluation can be solved through direct matching to a nearly equivalent training target.

The schemas of Unseen Functions are intentionally not removed from training entirely. They may appear as non-target candidates in the offered sets of ordinary training examples, but they are never the correct training answer. SIP can therefore learn the general task of comparing requests with supplied schemas without learning to select the held-out functions as target outputs. Each example contains an offered set of 4 to 24 functions, with median 14. Offered sets contain multiple Seen and Unseen schemas. This removes two potential shortcuts. If Unseen examples contained only unfamiliar schemas, the candidate composition itself could reveal the split. If the target were the only unfamiliar schema among otherwise Seen functions, it could instead be selected by elimination.

A.5 Out-of-Scope Requests

Out-of-Scope Requests require refusal because no suitable function is available for the current example. We construct two forms. The first requests functionality that cannot be satisfied by any function in the complete Function Surface. The corpus contains 1,156 examples of this type. The second requests functionality represented by a real function, but that function is absent from the offered set for the current example. The corpus contains 999 examples of this type. This case is particularly important because the request alone does not determine the correct response. A similar request may require execution when the function is available and refusal when it is absent. Both types use the same mixed offered-set construction as the rest of the benchmark. The presence of unfamiliar schemas therefore does not itself provide a cue that the correct output is refusal.

A.6 Training and Evaluation Partition

The function holdout and the example partition serve different purposes. The function holdout determines which functions may occur as training targets. The example partition determines which individual requests belong to training or evaluation. No example targeting an Unseen Function is assigned to training. Out-of-Scope examples occur in both partitions because refusal is a behavior that must be represented during training. For ordinary examples, partitioning is based on normalized request text. Duplicate or punctuation-equivalent requests therefore remain on the same side of the split. Globally unsupported requests require an additional safeguard. Multiple natural-language requests can be generated from the same underlying unsupported capability. If these were partitioned only by wording, one phrasing could appear during training and another during evaluation. We therefore assign all phrasings of the same underlying unsupported request type to the same partition. Evaluation consequently tests refusal on held-out request types rather than only on alternative phrasings of request types observed during training.

A.7 Quality Control

The quality-control procedure separates properties that can be checked deterministically from properties that require semantic judgement.

Structured-call validation. Every Vehicle Function Call used as a target is checked against its schema before language generation. The validator verifies function identity, required arguments, argument types, permitted values, numerical ranges, and applicable vehicle areas.

Target Leakage. Synthetic data can become artificially easy when the generated request directly exposes the identifier of its target function. We refer to this as *Target Leakage*. Requests are checked for literal function identifiers and distinctive combinations of identifier tokens. Ordinary argument values are not treated as leakage by themselves because a user may naturally need to specify a temperature, level, or mode. Requests that trigger the leakage criterion are regenerated or removed. The same leakage criterion is applied before evaluation scoring.

Independent semantic screening. We use Mistral-Small-3.2-24B-Instruct as an independent semantic screen. It belongs to a different model family from both the corpus writer and the SLMs evaluated in this paper. For examples with a Vehicle Function Call as the target, the screen checks whether the generated request is consistent with the structured call. For refusal examples, the relevant available functions are also considered when assessing whether refusal is appropriate. We do not treat this model-based screen as ground truth. Comparison against manual review showed that some semantic error judgements were unreliable, particularly for functions whose movement direction is represented through signed argument values. These judgements are therefore retained as diagnostics rather than used automatically to remove examples. Automatic filtering is restricted to leakage cases supported by the independent programmatic leakage check.

Verified Slice. We manually review a stratified Verified Slice containing 217 examples across 42 strata defined by evaluation split, function group, and argument structure. For each sampled example, the reviewer checks whether the request faithfully expresses its assigned Vehicle Function Call or refusal and whether another call could satisfy the same request equally well. The review finds 100.0% of the sampled examples faithful and 99.1% unambiguous. Of the 217 reviewed examples, 202 belong to the initial review pass. An additional 15 examples were reviewed after targeted corpus updates, and 9 of those had themselves been modified. We therefore treat the Verified Slice as evidence about the sampled corpus rather than as a fully blinded estimate or a guarantee over every generated example.

A.8 Evaluation Filtering

The counts in Table 1 describe the generated training and evaluation partitions before the final leakage filter is applied. Examples flagged by the supported leakage criterion are excluded before scoring. All reported experimental metrics therefore use 545 Seen, 2,136 Unseen, and 533 Out-of-Scope examples as their respective denominators.

A.9 Scope and Limitations

The benchmark evaluates English, Single-Turn Function Calling. It does not require dialogue history, changing vehicle state, or sequences of dependent Vehicle Function Calls. This scope isolates the effect of Function Surface representation from multi-turn planning and state tracking. The utterances are synthetically generated rather than collected from drivers. Programmatic validation establishes the legality of the structured targets. Leakage auditing, independent semantic screening, and manual review address complementary sources of language-level error. These checks do not establish that the synthetic utterance distribution matches naturally occurring in-vehicle requests. Evaluation on naturally collected requests, additional languages, and multi-turn interactions remains future work.

B Theoretical Analysis of In-Vehicle Function Calling

B.1 Task-Level Output Support

We formalize the difference between Functional Tokens (FTs) and Schema-in-Prompt (SIP) in terms of their task-level output support. Let Σ denote the tokenizer vocabulary and let Σ^* denote the set of finite token sequences over Σ . A *Function Class* is a finite set $\mathcal{F} = \{f_1, \dots, f_m\}$. Each function $f \in \mathcal{F}$ has a schema $S(f) \in \Sigma^*$ containing its name, natural-language description, argument names, argument types, and value constraints.

For an evaluation row i , let $x_i \in \Sigma^*$ denote the user request and $O_i \subseteq \mathcal{F}$ the set of functions offered for that vehicle. Let $\mathcal{C}(x_i) \subseteq \mathcal{F}$ denote the set of functions that can semantically satisfy request x_i . The target function-selection decision is $y_i \in O_i \cup \{\perp\}$, where $\perp$ denotes refusal or abstention. Under our task definition,

$$y_i = \perp \iff \mathcal{C}(x_i) \cap O_i = \emptyset.$$

Thus, an *Out-of-Scope Request* is one for which no function capable of satisfying the request is available in the offered set. This includes both requests unsupported by the vehicle function class and requests whose required function exists in $\mathcal{F}$ but is absent from O_i . This analysis isolates function identity selection; argument generation is omitted for clarity and does not change the output-support distinction considered below. A *Seen Function* is a function that appears as a target during training, whereas an *Unseen Function* is withheld as a training target but may still appear among the schemas offered at inference time.

Let $T \subseteq \mathcal{F}$ denote the set of Seen Functions, each of which is assigned a dedicated FT during training, and let

$$\tau : T \rightarrow \Sigma$$

be an injective mapping from each function to its dedicated token. Although the underlying language model generates sequences in Σ^* , the evaluated FT decoder maps valid FT outputs to function identities in T . Therefore, at the task level, the admissible FT output set is

$$\mathcal{Y}_{\text{FT}} = T \cup \{\perp\}.$$

Let $p_{\text{FT}}(y \mid x_i)$ denote the task-level output distribution induced by the language model together with this FT decoding rule. Then

$$p_{\text{FT}}(f \mid x_i) = 0 \quad \forall f \in \mathcal{F} \setminus T.$$

This is a statement about the support of the evaluated representation rather than about semantic similarity or the raw token-level language-model distribution. A request for an Unseen Function may be linguistically close to requests for Seen Functions, and the model could in principle generate ordinary tokens resembling its name. However, because no dedicated FT maps to $f \notin T$, such a sequence is not decoded as a valid prediction of f under the evaluated FT representation. Consequently, exact Unseen Function accuracy is zero by construction. The informative measurement is therefore the observed failure mode: (i) refusal, (ii) prediction of an incorrect Seen Function, or (iii) an invalid or unparseable output.

Schema-in-Prompt instead constructs the admissible function set from the schemas supplied at inference time. Let

$$S(O_i) \in \Sigma^*$$

denote the serialized representation of the schemas associated with the functions in O_i . If

$$O_i = \{f_{i,1}, \dots, f_{i,r_i}\},$$

then, for the ordering used by the prompting procedure,

$$\mathcal{S}(O_i) = \mathcal{S}(f_{i,1}) \parallel \mathcal{S}(f_{i,2}) \parallel \dots \parallel \mathcal{S}(f_{i,r_i}),$$

where $\parallel$ denotes concatenation together with the required schema separators and formatting tokens.

As with FT, we express the SIP decision rule at the task level, after mapping generated token sequences to valid function identities or refusal:

$$\hat{y}_{\text{SIP}}(x_i, O_i) = \arg \max_{y \in O_i \cup \{\perp\}} p_{\text{SIP}}(y \mid x_i, \mathcal{S}(O_i)),$$

where p_{SIP} denotes the task-level output distribution induced by the language model and the SIP parsing rule.

The admissible SIP function set is therefore row-specific. If an Unseen Function $f \notin T$ is included in O_i , it remains a valid task-level output even though it was never observed as a training target. This difference in output support explains the qualitative difference in Unseen Function behavior. Under the evaluated FT representation, a function outside T has no corresponding valid Functional Token and therefore cannot be recovered as the exact target. Under SIP, the same function can enter the task-level output support through its schema and may be selected by interpreting that schema at inference time.

The same distinction affects refusal behavior. The correct decision depends on whether any function capable of satisfying the request belongs to the offered set:

$$\mathcal{C}(x_i) \cap O_i = \emptyset \implies y_i = \perp.$$

At the model level, this can be viewed through a compatibility score $s_\theta(x, f)$ between request x and function f . For example, a selective decision rule may reject when

$$\max_{f \in O_i} s_\theta(x_i, f) < \gamma,$$

for some rejection threshold γ . This connects the task to classical reject rules and modern selective classification (Chow, 1970; Geifman and El-Yaniv, 2017). In the evaluated FT formulation, O_i is not explicitly exposed to the model. Consequently, an FT model may emit a Seen Function that is compatible with the request but unavailable in the current row. SIP explicitly provides O_i through its serialized schemas, allowing both function selection and refusal to condition on the functions actually available.

This additional flexibility incurs a computational cost. Let $|z|$ denote the token length of sequence z , and define

$$L_{\text{SIP},i} = |x_i| + |\mathcal{S}(O_i)| + c_{\text{SIP}},$$

where c_{SIP} denotes fixed prompt and output-formatting overhead. Similarly, the FT input length is approximately

$$L_{\text{FT},i} = |x_i| + c_{\text{FT}},$$

where c_{FT} denotes the corresponding fixed overhead. Since

$$|\mathcal{S}(O_i)| \approx \sum_{f \in O_i} |\mathcal{S}(f)|$$

grows with both the number and length of the offered schemas, $L_{\text{SIP},i}$ can be substantially larger than $L_{\text{FT},i}$.

For a standard Transformer with dense self-attention, attention-score computation during prompt prefill scales quadratically with input sequence length (Vaswani et al., 2017). Thus, considering the attention component alone, SIP incurs approximately

$$\mathcal{O}((L_{\text{SIP},i})^2)$$

prefill computation, while key-value cache storage grows linearly with the cached sequence length. With key-value caching, the attention computation for each subsequent autoregressive decoding step is approximately linear in the existing context length. Therefore, the longer schema-augmented SIP prompts increase both prompt-processing work and cache requirements relative to FT. This scaling behavior is consistent with our proxy-device measurements: SIP extends task-level output support to Unseen Functions, but the additional schema tokens increase time to first call relative to FT fine-tuning.

B.2 Generalization through Schema-in-Prompt Learning

We next consider how additional training examples can improve Schema-in-Prompt (SIP) generalization. The notation in the preceding subsection describes individual task rows (x_i, O_i, y_i) . To reason about training statistically, let (X, O, Y) denote the corresponding random task variables, whose realizations have the same domains:

$$X \in \Sigma^*, \quad O \subseteq \mathcal{F}, \quad Y \in O \cup \{\perp\}.$$

Let P_{tr} denote the training distribution and let

$$D_n = \{(x_i, O_i, y_i)\}_{i=1}^n$$

be a training set of n examples sampled from P_{tr} . For the task-level analysis, let

$$\ell(\hat{y}, y) = \mathbf{1}\{\hat{y} \neq y\}$$

denote the zero-one loss, irrespective of the token-level objective used to train the language model. For a task-level predictor h_θ , define the training-distribution population risk and empirical risk as

$$R_{\text{tr}}(\theta) = \mathbb{E}_{(X, O, Y) \sim P_{\text{tr}}} [\ell(h_\theta(X, O), Y)],$$

and

$$\hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_\theta(x_i, O_i), y_i).$$

Under standard i.i.d. sampling and uniform-convergence assumptions, generalization bounds take the schematic form

$$R_{\text{tr}}(\hat{\theta}_n) \leq \hat{R}_n(\hat{\theta}_n) + \epsilon_n(\mathcal{H}, \delta),$$

where $\hat{\theta}_n$ denotes the learned parameters, $\mathcal{H}$ is the hypothesis class, $\delta \in (0, 1)$ is a confidence parameter, and $\epsilon_n(\mathcal{H}, \delta)$ is a complexity-dependent estimation term. For finite-capacity hypothesis classes, such terms commonly exhibit dependence of the form

$$\epsilon_n(\mathcal{H}, \delta) = \mathcal{O}\left(\sqrt{\frac{\mathcal{C}(\mathcal{H}) + \log(1/\delta)}{n}}\right),$$

for an appropriate capacity measure $\mathcal{C}(\mathcal{H})$ (Shalev-Shwartz and Ben-David, 2014, Ch. 6). We use this expression only as a learning-theoretic motivation for training-time example scaling rather than as a model-specific generalization guarantee for the neural predictor used in our experiments. In particular, increasing n can reduce the estimation component under the assumptions above, but the bound neither guarantees monotonic empirical improvement nor, by itself, establishes generalization to functions withheld as training targets.

The Unseen Function setting introduced in Appendix B.1 requires an additional form of transfer. Let P_{te} denote the evaluation distribution. In the held-out-function evaluation, P_{te} may contain rows whose target $f_u \in \mathcal{F} \setminus T$ never appears as a training target under P_{tr} . Thus, the relevant SIP generalization mechanism cannot be explained solely as estimation of a fixed mapping from requests to training-time function identities. Instead, SIP can exploit a relation between request language and schema language that is shared across functions.

To make this intuition explicit, consider the compatibility score introduced in Appendix B.1 under the following factorization:

$$s_\theta(x, f) = \phi_\theta(x)^\top \psi_\theta(S(f)),$$

where

$$\phi_\theta : \Sigma^* \rightarrow \mathbb{R}^d, \quad \psi_\theta : \Sigma^* \rightarrow \mathbb{R}^d$$

embed a request and a function schema, respectively. This factorization is an analytical abstraction of request-schema compatibility rather than a claim about the internal architecture of the autoregressive language model.

Generalizing the fixed threshold γ used illustratively in Appendix B.1, function selection with a possibly context-dependent reject threshold can be expressed as

$$h_\theta(x, O) = \begin{cases} \arg \max_{f \in O} s_\theta(x, f), & \max_{f \in O} s_\theta(x, f) \geq \gamma_\theta(x, O), \\ \perp, & \max_{f \in O} s_\theta(x, f) < \gamma_\theta(x, O), \end{cases}$$

where $\gamma_\theta(x, O)$ represents a possibly context-dependent rejection threshold. As in Appendix B.1, this is a task-level abstraction of the model’s function-selection and refusal behavior.

Under this view, additional training examples can improve SIP insofar as they improve request–schema relations that transfer beyond the specific function identities observed as training targets. Paraphrase diversity can reduce sensitivity to surface wording. Examples spanning different argument values or constraints can help distinguish the conditions under which otherwise similar functions should be selected or refused. Examples containing competing offered functions provide negative comparisons, requiring the model to distinguish schemas that may share vocabulary but correspond to different calls.

This provides a mechanism through which Unseen Function accuracy can improve even when the held-out function never appears as a training target. In particular, training on Seen Functions can improve parameters that encode correspondences between request language and schema descriptions. At evaluation time, the same learned relation can then be applied to the schema $S(f_u)$ of an Unseen Function $f_u \notin T$, provided that $f_u \in O_i$. Such transfer is enabled by the schema-mediated representation, but it is not guaranteed merely by increasing the number of training examples: the added examples must contain information that improves relations relevant to the evaluation distribution.

FTs behave differently under training-time example scaling because their task-level output support remains fixed. Recall that

$$\mathcal{Y}_{\text{FT}} = T \cup \{\perp\}.$$

For a Seen Function $f \in T$, additional training examples can improve discrimination among the existing outputs. They may also improve refusal behavior when out-of-scope status can be inferred from request-only cues represented during training. For an Unseen Function $f_u \notin T$, however, additional examples over the existing FT representation leave

$$f_u \notin \mathcal{Y}_{\text{FT}} \quad \text{and hence} \quad p_{\text{FT}}(f_u | x) = 0.$$

Thus, additional examples cannot make f_u a valid task-level FT output without extending the FT representation and training the model to associate a new dedicated token with that function. Moreover, because O is not explicitly exposed in the evaluated FT formulation, additional examples alone cannot enable the FT model to condition its refusal decision on the row-specific absence of an otherwise compatible function.

SIP instead couples its admissible function set to the offered set O . Consequently, a previously unseen function can become a valid task-level output simply by supplying its schema at evaluation time. Training-time example scaling can then improve two different aspects of SIP performance: estimation over patterns represented in the training distribution and transfer of learned request–schema relations to functions that were not observed as training targets. The latter distinguishes schema-mediated generalization from learning to classify more accurately among a fixed set of outputs.

B.3 Risk–Cost Trade-offs of In-Context Demonstrations

Although our experiments do not vary the number of in-context demonstrations, this subsection extends the preceding analysis to characterize the more general risk–cost trade-off that arises when additional examples are supplied through the inference context.

We now consider examples supplied at inference time rather than during parameter training. Let

$$\mathbf{E}_k = (e_1, \dots, e_k), \quad e_j = (x'_j, O'_j, y'_j),$$

denote an ordered sequence of k in-context demonstrations. Let

$$\mathcal{D}(\mathbf{E}_k) \in \Sigma^*$$

denote their complete serialization in the prompt, including the requests, offered schemas, target outputs, and required formatting. The sequence is ordered because demonstration selection and ordering can affect in-context predictions (Liu et al., 2022; Rubin et al., 2022; Lu et al., 2022). Using the notation of the preceding subsections, the SIP decision for evaluation row i becomes

$$\hat{y}_i^{(k)} = \arg \max_{y \in O_i \cup \{\perp\}} p_{\text{SIP}}(y | x_i, \mathcal{S}(O_i), \mathcal{D}(\mathbf{E}_k)).$$

The model parameters remain fixed; only the conditioning context changes. This is the inference-only adaptation commonly studied as in-context learning (Brown et al., 2020; Xie et al., 2021; Akyürek et al., 2023; von Oswald et al., 2023). Prior work has shown that demonstrations can provide information about the input distribution, label space, output format, and other task cues (Min et al., 2022).

Within the compatibility abstraction introduced above, conditioning on $\mathbf{E}_k$ induces a context-dependent score $s_{\theta, \mathbf{E}_k}$ and rejection threshold $\gamma_{\theta, \mathbf{E}_k}$. For a call-bearing row $y_i \in O_i$, define the decision margin

$$M_{i,k}^{\text{call}} = s_{\theta, \mathbf{E}_k}(x_i, y_i) - \max \left\{ \gamma_{\theta, \mathbf{E}_k}(x_i, O_i), \max_{f \in O_i \setminus \{y_i\}} s_{\theta, \mathbf{E}_k}(x_i, f) \right\},$$

where the maximum over an empty set is defined as $-\infty$. A positive margin means that the target function both outranks every competing offered function and exceeds the rejection threshold. For a refusal row $y_i = \perp$, define

$$M_{i,k}^{\text{ref}} = \gamma_{\theta, \mathbf{E}_k}(x_i, O_i) - \max_{f \in O_i} s_{\theta, \mathbf{E}_k}(x_i, f).$$

A positive refusal margin means that every offered function falls below the rejection threshold. Under this abstraction, demonstrations can improve a particular prediction by increasing the corresponding decision margin. There is no requirement, however, that an arbitrary additional demonstration increases that margin.

This view is complementary to information-theoretic analyses of in-context learning. For example, Jeon et al. (2024, Theorem 4.7) derive an upper bound on expected in-context log-loss containing irreducible, meta-estimation, and in-context estimation terms. Their result provides a theoretical account of how information supplied through the context can reduce estimation uncertainty under their assumptions. It does not imply that arbitrary demonstrations monotonically improve prediction, nor does it directly account for demonstration selection, ordering, distraction, or deployment latency. We therefore separate the predictive effect of demonstrations from their computational cost.

Using the evaluation distribution P_{te} defined above, let the task-level evaluation risk of a predictor h be

$$R_{\text{te}}(h) = \mathbb{E}_{(X, O, Y) \sim P_{\text{te}}} [\ell(h(X, O), Y)].$$

Let h_0 denote the SIP predictor without in-context demonstrations, and let $h_{\mathbf{E}}$ denote the predictor conditioned on demonstration sequence $\mathbf{E}$. For a demonstration budget k , let $\mathcal{A}_k$ denote the set of admissible ordered demonstration sequences containing at most k examples, including the empty sequence. Define the best achievable evaluation risk under this budget as

$$R_{\text{te}}^*(k) = \inf_{\mathbf{E} \in \mathcal{A}_k} R_{\text{te}}(h_{\mathbf{E}}).$$

For a finite admissible demonstration pool, the infimum can be replaced by a minimum. Because the empty sequence belongs to $\mathcal{A}_k$,

$$R_{\text{te}}^*(k) \leq R_{\text{te}}(h_0).$$

Define the attainable predictive gain at budget k as

$$\Delta_{\text{gain}}(k) = R_{\text{te}}(h_0) - R_{\text{te}}^*(k) \geq 0.$$

For the particular demonstration sequence $\mathbf{E}_k$ supplied to the model, define its context gap as

$$\Delta_{\text{gap}}(\mathbf{E}_k) = R_{\text{te}}(h_{\mathbf{E}_k}) - R_{\text{te}}^*(k) \geq 0.$$

The context gap measures the excess predictive risk of the selected context relative to the best admissible context under the same demonstration budget. It may reflect suboptimal demonstration selection or ordering, irrelevant or conflicting examples, and other context-induced effects. It should not be interpreted solely as a penalty arising from sequence length.

By construction,

$$R_{\text{te}}(h_{\mathbf{E}_k}) = R_{\text{te}}(h_0) - \Delta_{\text{gain}}(k) + \Delta_{\text{gap}}(\mathbf{E}_k).$$

Therefore, the selected demonstrations improve predictive risk relative to the no-demonstration predictor exactly when

$$R_{\text{te}}(h_{\mathbf{E}_k}) < R_{\text{te}}(h_0) \iff \Delta_{\text{gain}}(k) > \Delta_{\text{gap}}(\mathbf{E}_k).$$

The use of at most k demonstrations gives a useful monotonicity property. Since

$$\mathcal{A}_k \subseteq \mathcal{A}_{k+1},$$

it follows that

$$R_{\text{te}}^*(k+1) \leq R_{\text{te}}^*(k),$$

or equivalently,

$$\Delta_{\text{gain}}(k+1) \geq \Delta_{\text{gain}}(k).$$

Thus, increasing the demonstration budget cannot worsen the *best achievable* predictive risk, because every context admissible under budget k remains admissible under budget $k+1$. This does not imply monotonic performance for the particular context actually used. The observed risk

$$R_{\text{te}}(h_{\mathbf{E}_k})$$

may increase or decrease with k because the context gap can vary with demonstration selection, relevance, and ordering.

In-context demonstrations also incur a computational cost. Using the input-length notation of Appendix B.1, define

$$C_{i,k} = |x_i| + |\mathcal{S}(O_i)| + |\mathcal{D}(\mathbf{E}_k)| + c_{\text{SIP}}.$$

For a standard Transformer with dense self-attention, attention-score computation during prompt prefill scales quadratically with the input sequence length,

$$\mathcal{O}(C_{i,k}^2),$$

while other components of Transformer computation have different scaling behavior (Tay et al., 2022). This describes the sequence-length dependence of dense self-attention rather than an exact model of wall-clock latency.

To keep predictive quality and deployment cost distinct, let

$$K_{\text{te}}(\mathbf{E}) = \mathbb{E}_{(X,O,Y) \sim P_{\text{te}}} [\kappa(X, O, \mathbf{E})]$$

denote the expected inference cost associated with demonstration sequence $\mathbf{E}$, where κ may denote a theoretical computation proxy or a measured device quantity such as time to first call. Define the additional inference cost relative to the no-demonstration setting as

$$\Delta_{\text{cost}}(\mathbf{E}_k) = K_{\text{te}}(\mathbf{E}_k) - K_{\text{te}}(\emptyset).$$

If predictive risk and deployment cost are to be optimized jointly, they can be combined explicitly through

$$J_{\text{te}}(\mathbf{E}) = R_{\text{te}}(h_{\mathbf{E}}) + \lambda K_{\text{te}}(\mathbf{E}), \quad \lambda \geq 0,$$

where λ converts inference cost into the units of the deployment objective and specifies the application-dependent trade-off between predictive quality and computational cost. Substituting the definitions above gives

$$J_{\text{te}}(\mathbf{E}_k) - J_{\text{te}}(\emptyset) = -\Delta_{\text{gain}}(k) + \Delta_{\text{gap}}(\mathbf{E}_k) + \lambda \Delta_{\text{cost}}(\mathbf{E}_k).$$

Hence the selected demonstrations improve the joint deployment objective exactly when

$$\Delta_{\text{gain}}(k) > \Delta_{\text{gap}}(\mathbf{E}_k) + \lambda \Delta_{\text{cost}}(\mathbf{E}_k).$$

This decomposition separates three effects of in-context demonstrations: the predictive gain made attainable by access to demonstrations, the predictive gap associated with the particular demonstrations supplied to the model, and the additional deployment cost of the longer context. It also clarifies why the empirically useful number of demonstrations need not increase monotonically. A larger demonstration budget expands the set of contexts from which a useful prompt can be constructed, but a particular longer context may provide little additional predictive benefit, may introduce a larger context gap, and incurs additional prompt-processing cost. For an in-vehicle assistant operating under latency and compute constraints, the relevant operating point is therefore determined by the balance between predictive improvement and the cost of additional context.

C Experimental Details and Additional Results

This appendix provides the implementation details, evaluation definitions, and additional analyses supporting Sections 4 and 5. We first describe the training and inference setup, then define the reported metrics and aggregation procedures. We subsequently provide the complete main-grid results, paired statistical tests, refusal decomposition, ablations, and Device Proxy measurements.

C.1 Training and Inference Details

All model–representation pairs use the same training examples and optimization recipe, with three independently trained seeds per configuration. Training uses three epochs, a learning rate of 2×10^{-5} , batch size 16, a maximum sequence length of 4096 tokens, and a warmup ratio of 0.03. The 270M and Qwen3-0.6B models use full fine-tuning. Qwen3-1.7B uses LoRA for both FT and SIP in the main experimental grid; the corresponding fully fine-tuned SIP configuration is evaluated separately in Appendix C.6.

Inference uses greedy decoding with at most 128 generated tokens. FT and SIP predictions are parsed using matched criteria so that neither representation benefits from more permissive handling of formatting deviations.

C.2 Evaluation Metrics and Aggregation

All model configurations are evaluated on the scored benchmark examples defined in Appendix A.8. The same evaluation examples are used for all models and seeds within each split.

For Seen and Unseen Functions, *accuracy* is the fraction of scored examples for which the parsed prediction exactly matches the target Vehicle Function Call, including its function and arguments, under the common evaluation criteria. For Out-of-Scope Requests, accuracy is the fraction for which the model correctly refuses rather than producing a Vehicle Function Call.

For Unseen Function requests, we additionally report *Execution*, the fraction of scored examples for which the model produces a parsed Vehicle Function Call to a valid function. Exact accuracy is therefore a subset of Execution, and the difference between Execution and accuracy captures *incorrect execution*: cases in which a Vehicle Function Call is produced but does not exactly match the target call. Other unsuccessful outcomes, such as refusal or invalid output, are not counted as Execution. For Out-of-Scope Requests, *Over-trigger* is the fraction of scored examples for which the model produces a Vehicle Function Call when the correct behavior is refusal.

Unless otherwise stated, accuracy and behavioral rates are first computed separately for each of the three independently trained seeds and then averaged across seeds. Statistical tests instead operate directly on paired example-level correctness for each seed, as described in Appendix C.4. Device Proxy measurements use the separate aggregation procedure described in Appendix C.7.

C.3 Complete Main-Grid Results

Table 3 provides the complete numerical accuracy results for all model–representation pairs in the main experimental grid. These values underlie the Seen Function and Out-of-Scope results visualized in Figure 2, as well as the Unseen Function accuracy shown in the left panel of Figure 3. The figures emphasize the main performance trends, while Table 3 provides the exact mean accuracies used for comparison.

C.4 Paired Statistical Tests

Because the compared configurations are evaluated on the same examples, differences in aggregate accuracy alone do not capture whether the models succeed and fail on the same requests. We therefore use two-sided exact McNemar tests on paired example-level correctness. For each seed, the test considers only examples on which the two compared configurations differ in correctness. Its *p*-value quantifies the evidence against the null hypothesis that the two directions of disagreement are equally likely. We use a significance level of 0.05. A small *p*-value therefore indicates a systematic paired difference between the two configurations. A non-significant result means that such

Representation	Model	Seen	Unseen	Out-of-Scope
Functional Token	Qwen3-1.7B	95.5	0.0	82.6
	Qwen3-0.6B	97.9	0.0	83.1
	FunctionGemma-270M	97.1	0.0	81.4
	Gemma 3-270M	97.5	0.0	81.6
Schema-in-Prompt	Qwen3-1.7B	98.1	84.1	95.7
	Qwen3-0.6B	98.7	59.3	98.4
	FunctionGemma-270M	97.2	18.6	97.3
	Gemma 3-270M	97.6	15.6	96.9

Table 3: Accuracy (%) averaged over three seeds for the complete main experimental grid. Out-of-Scope accuracy denotes correct refusal. Qwen3-1.7B uses LoRA for both representations, while the smaller models use full fine-tuning.

a difference is not detected and does not establish statistical equivalence. The tests are performed separately for the three training seeds.

Table 4 reports the comparisons used to support two claims in Section 5. The FunctionGemma-270M SIP versus Qwen3-1.7B SIP comparison tests whether the substantially larger configuration improves Seen Function performance. The FunctionGemma-270M SIP versus Gemma 3-270M SIP comparison instead holds model scale approximately fixed and tests whether prior function-calling specialization changes Seen or Unseen performance.

Comparison (Schema-in-Prompt)	Split	Seed 0	Seed 1	Seed 2
FunctionGemma-270M vs Qwen3-1.7B	Seen	0.607	0.267	0.238
FunctionGemma-270M vs Gemma 3-270M	Seen	0.453	1.000	0.581
FunctionGemma-270M vs Gemma 3-270M	Unseen	4.5×10^{-4}	5.6×10^{-6}	4.8×10^{-6}

Table 4: Two-sided exact McNemar p -values for paired SIP comparisons. Each column corresponds to one independently trained seed, and each test uses identical evaluation examples for the two configurations. Values below 0.05 indicate a detected paired difference at the stated significance level.

For FunctionGemma-270M versus Qwen3-1.7B, no significant difference is detected on Seen Functions in any seed. This provides no evidence that the larger configuration improves Seen performance in this comparison, but it does not establish equivalence.

For FunctionGemma-270M versus Gemma 3-270M, no significant difference is detected on Seen Functions. On Unseen Functions, FunctionGemma has higher accuracy and the paired difference is significant in every seed. This provides evidence that prior function-calling specialization benefits Unseen Function interpretation for this same-scale model pair. We do not generalize this result beyond the evaluated pair.

C.5 Out-of-Scope Refusal Decomposition

Out-of-Scope Requests arise for two reasons. The requested functionality may be absent from the complete Function Surface, or the required function may exist in the Function Surface but be absent from the offered set for the current example. The latter case directly tests whether refusal depends on recognizing current function availability, so we analyze these examples separately.

We divide these requests according to whether the requested but unavailable function is a Seen Function and therefore has a trained Functional Token. The grouping is determined from the benchmark Function Surface partition and not from model predictions. The n shown for each group in Table 5 is the number of unique scored evaluation requests in that group. It is not pooled across seeds. Every model and each of its three seeds are evaluated on the same n requests. The reported refusal rates are the mean of the three per-seed correct-refusal rates.

Unavailable-function group	Model	FT refusal	SIP refusal
FT has a token ($n = 92$)	Gemma 3-270M	2.9	85.5
	FunctionGemma-270M	4.7	89.5
	Qwen3-0.6B	4.7	93.5
	Qwen3-1.7B	5.1	81.2
FT has no token ($n = 164$)	Gemma 3-270M	99.2	99.8
	FunctionGemma-270M	98.8	99.0
	Qwen3-0.6B	99.6	99.6
	Qwen3-1.7B	97.6	97.8

Table 5: Correct-refusal rates (%) for Out-of-Scope Requests whose required function exists in the Function Surface but is absent from the offered set. n denotes the number of unique scored evaluation requests in each availability group. Rates are averaged over three seeds.

Model	SIP configuration	Seen	Unseen	Out-of-Scope
Qwen3-1.7B	LoRA $r=16$ (main grid)	98.1	84.1	95.7
	Full fine-tune	99.0	75.4	98.6
FunctionGemma-270M	Uniform JSON Schema (main grid)	97.2	18.6	97.3
	Native tool-calling format	97.4	24.9	94.8

Table 6: SIP ablations for the Qwen3-1.7B adaptation method and FunctionGemma prompt format. Accuracy (%) is averaged over three seeds. Each ablation is shown with its corresponding main-grid configuration.

When FT has a trained token for the unavailable function, its correct-refusal rate is substantially lower than that of SIP across all evaluated models. In these examples, the requested function belongs to the learned FT output space, but the evaluated FT input does not expose the row-specific offered set. FT therefore cannot directly condition its decision on the function’s absence.

When FT has no trained token for the unavailable function, both representations refuse much more reliably. The aggregate Out-of-Scope accuracy therefore combines two qualitatively different cases. The large FT–SIP refusal difference is concentrated in examples where FT can produce the requested function but cannot observe that it is unavailable.

C.6 Adaptation and Prompt-Format Ablations

Two ablations test experimental choices that could otherwise confound interpretation of the main grid.

Qwen3-1.7B adaptation. The main Qwen3-1.7B configuration uses LoRA, whereas the smaller models use full fine-tuning. We therefore repeat Qwen3-1.7B SIP using full fine-tuning. Its Unseen Function accuracy decreases relative to the LoRA main-grid configuration but remains above Qwen3-0.6B SIP. The within-Qwen ordering reported in the main results therefore remains under either adaptation method. The experiment does not establish that LoRA is generally superior to full fine-tuning. Full fine-tuning also improves Out-of-Scope accuracy relative to the LoRA configuration, indicating that adaptation choice contributes to the 1.7B refusal behavior.

FunctionGemma prompt format. The main grid uses a common SIP representation across model families. FunctionGemma also provides a native function-calling format, so we evaluate that format while holding the corpus and evaluation examples fixed. The native format improves Unseen Function accuracy but reduces Out-of-Scope accuracy. This indicates that the uniform SIP format suppresses some of FunctionGemma’s schema-reading capability while the native format introduces a refusal trade-off. The improvement does not remove the larger gap between the 270M and Qwen SIP configurations.

C.7 Device Proxy Measurements

All latency and memory measurements are Device Proxy measurements. They are collected on a Xeon 8580 CPU using four threads under Q4_K_M quantization. No vehicle hardware is used. The same measurement procedure is applied to FT and SIP so that the comparison primarily reflects their relative inference cost rather than absolute in-vehicle latency.

Representation	Model	Prompt tokens	Time to first call (ms)	Peak RSS (MB)
Functional Token	Qwen3-1.7B	14	104 (91–360)	2683 (2682–2691)
	Qwen3-0.6B	14	141 (124–143)	1216 (1215–1338)
	FunctionGemma-270M	16	82 (77–90)	408 (407–411)
	Gemma 3-270M	16	87 (64–88)	407 (405–407)
Schema-in-Prompt	Qwen3-1.7B	1,799	10206 (10039–10319)	6514 (6512–6516)
	Qwen3-0.6B	1,799	5593 (5254–5666)	5126 (5126–5126)
	FunctionGemma-270M	1,921	2338 (2290–2349)	1208 (1208–1209)
	Gemma 3-270M	1,921	2277 (2248–2341)	1208 (1208–1209)

Table 7: Device Proxy inference measurements for the main experimental grid. Time to first call and peak resident memory are reported as the median across three measurement runs, with the observed minimum–maximum range in parentheses. All measurements use the same Xeon 8580 CPU, four threads, and Q4_K_M quantization. These values do not represent latency or memory use on vehicle hardware.

Prompt tokens is the number of input tokens processed before generation begins. *Time to first call* is the elapsed Device Proxy time until the generated output first forms a parseable Vehicle Function Call. *Peak RSS* is the maximum resident set size observed during the measurement run. Time to first call and peak RSS are summarized by the median over the three runs, with the observed range shown in parentheses.

SIP processes substantially longer prompts because the offered schemas are serialized in its input, whereas FT uses a compact request-only representation. The increased prompt-processing work accounts for most of the SIP latency difference and is consistent with the sequence-length analysis in Section 3. The especially large relative time difference for Qwen3-1.7B should not be interpreted as a scaling law. Its FT configuration reaches a parseable call using fewer decoded tokens than Qwen3-0.6B FT, reducing the FT time that forms the denominator of the ratio. Peak memory follows a different pattern because the model weights account for a larger fraction of resident memory as model size increases.